From Purchases to Communities:

How User Activity Shapes Product Networks Built from Purchase Data

Tong Zhou Sia Xie Yilin Jiang Mingjiao Diao

STAT 236, Fall 2026

Abstract

Product networks built from purchase data link two products whenever the same customer buys both. A customer who buys d products contributes $\frac{d(d-1)}{2}$ links, so very active customers can dominate the network. A common response is to downweight them; we ask when this helps. In a simulation with known product categories, we compare raw co-purchase counts, Jaccard similarity, inverse-activity weighting, and a normalized SVD of the user–product matrix while varying how unevenly activity is spread, whether active users have category preferences, and how strong those preferences are. When active users focus on one category they carry most of the signal, then downweighting them is costly: with the most concentrated activity, weighting raises the misclustering rate from 13% to 49%. When active users buy indiscriminately and preferences are weak, no method recovers the categories; with stronger preferences, weighting lowers the error by up to 52 percentage points. A second design that holds informative purchases fixed confirms this: added preference-free heavy buyers drive raw counts to chance while weighting stays near its baseline. Activity is not a proxy for noise, and whether to downweight it depends on what active users buy.

1 Introduction

Purchase records form a bipartite network between users and products. A standard way to analyze such data is to *project* it onto the products, connecting two products when they share buyers, and then to cluster the resulting network (von Luxburg, 2007; Ke, 2026). The clusters describe groups of products that tend to be bought by the same people.

The projection has a well-known weakness: a user who buys d distinct products creates $\frac{d(d-1)}{2}$ product pairs, so a handful of heavy buyers can supply most of the edge weight. The common remedy, going back to the collaboration networks of Newman (2001), is to give each user's pairs weight $\frac{1}{d-1}$, which makes a user's total contribution grow only linearly.

Whether this is a good idea depends on what active users buy, which the weighting ignores (Figure 1). If a heavy buyer spreads purchases across categories, most of their links cross category boundaries and downweighting removes noise. If the same buyer concentrated on one category, downweighting would discard the strongest evidence in the data. We ask three questions:

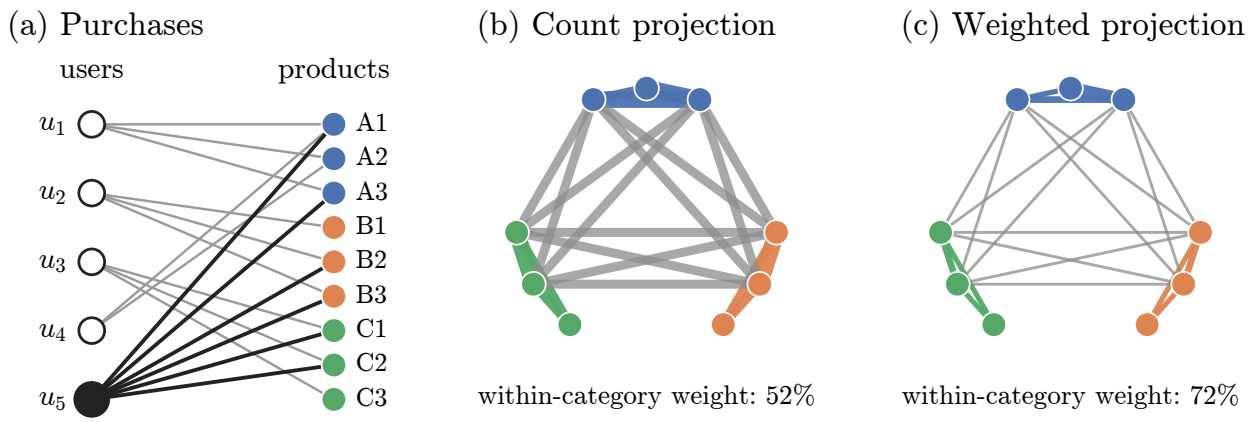


Figure 1: A toy example. (a) Purchases: users u_1–u_4 buy within a single category; u_5 buys six products across all three. (b) Under counts, every pair created by u_5 has full weight and cross-category edges (grey) dominate. (c) Inverse-activity weighting gives each of u_5's pairs weight $1/5$, and the within-category share of edge weight rises from 52% to 72%. If u_5 instead bought six products from one category, the same weighting would shrink useful edges.

Q1. How do activity heterogeneity and the breadth of active users' preferences interact with the choice of projection weights?

Q2. When does inverse-activity weighting improve on raw counts and Jaccard similarity?

Q3. How do the projections compare with a normalized SVD applied directly to the user–product matrix?

We answer these questions with a simulation in which product categories are known and expected edge weights can be computed exactly. Making heavy buyers indiscriminate both adds noise and, if the amount of data is held fixed, removes informative purchases, so we use two complementary designs: Design 1 holds total purchases fixed and Design 2 holds the informative purchases fixed. Together they cover 54 conditions and 5,400 independent data sets, and all four methods are applied to the same data sets.

Related work. Besides Newman (2001), Zhou et al. (2007) discount active users in a resource-allocation projection for recommendation. The SVD baseline is motivated by bipartite spectral co-clustering (Dhillon, 2001) and its extension to directed graphs (Rohe et al., 2016), though we do not implement either algorithm exactly.

2 Model and methods

2.1 A simple purchase model

Let $B \in \{0,1\}^{n\times p}$ record purchases, with $B_{ui} = 1$ if user u bought product i. We use $n = 1{,}000$ users and $p = 300$ products divided into $K = 3$ categories of $s = p/K = 100$ products. Each user has a preferred category, drawn uniformly at random, and buys d_u distinct products one at a time without replacement. At each draw, a remaining product

in the preferred category has weight $r_u \geq 1$ and every other product has weight 1. With $r_u = 2$, about half of a user's purchases fall in the preferred category; with $r_u = 1$ the user has no preference at all. Users are generated independently of one another. Within a user, purchases are dependent, because the d_u products are distinct and each draw changes the weights of the remaining products. Section A gives the full procedure and a worked example.

2.2 Three product networks

Let $S_{ij} = \sum_u B_{ui}B_{uj}$ be the number of shared buyers of products $i \neq j$ and $b_i = \sum_u B_{ui}$ the number of buyers of i. We compare

$$W_{ij}^{\text{count}} = S_{ij}, \qquad W_{ij}^{\text{jac}} = \frac{S_{ij}}{b_i + b_j - S_{ij}}, \qquad W_{ij}^{\text{wtd}} = \sum_{u:\, d_u \geq 2} \frac{B_{ui}B_{uj}}{d_u - 1}.$$

All diagonals are set to zero and no edges are thresholded. For each network W with degree matrix D, we take the eigenvectors of $D^{-1/2}WD^{-1/2}$ belonging to the K largest eigenvalues, normalize each row of the resulting embedding, and run k-means with 20 random starts.

As a fourth method we skip the projection. We form $M = D_u^{-1/2}BD_p^{-1/2}$, where D_u and D_p hold user and product degrees, and cluster the products using the top K right singular vectors of M, with the same row normalization and k-means step. Since

$$M^\top M = D_p^{-1/2}\, B^\top D_u^{-1} B\, D_p^{-1/2}, \tag{1}$$

this baseline is itself a projection that weights each user by $1/d_u$, close to the $1/(d_u - 1)$ of WEIGHTED. The main differences are that it keeps the diagonal of $M^\top M$ and that users with a single purchase still enter it. All four methods are given the true number of categories, $K = 3$.

2.3 Expected edge weights and user influence

Because products within a category are exchangeable and each user's preferred category is uniform and independent of everything else, the probability that user u buys a particular pair of distinct products depends only on whether the two products share a category. Writing C_{uk} for the number of products u buys from category k,

$$q_u^{\text{same}} = \frac{\mathbb{E}[C_{uk}(C_{uk} - 1)]}{s(s-1)}, \qquad q_u^{\text{diff}} = \frac{\mathbb{E}[C_{uk}C_{ul}]}{s^2} \quad (k \neq l).$$

Summing q_u over users gives the expected COUNT weight of a within- or between-category pair; dividing each term by $d_u - 1$ gives the WEIGHTED expectation. We compute the required moments exactly with a short dynamic program over the sequential sampling scheme (Section B). For JACCARD we use the ratio of expected shared buyers to expected union size, an approximation that agrees with the simulated means to within 0.23% in every condition (Section C).

Two facts follow directly. A user with $r_u = 1$ has $q_u^{\text{same}} = q_u^{\text{diff}}$ and adds no expected within-minus-between contrast, and a user with $d_u \geq 2$ contributes total weight $d_u(d_u - 1)/2$

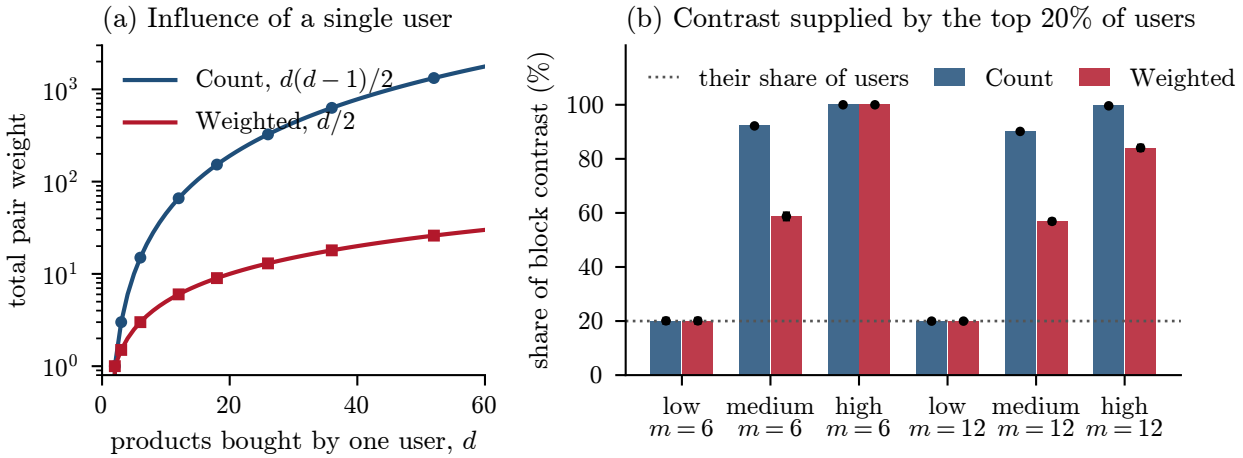


Figure 2: (a) Total pair weight contributed by one user who buys d products; markers show the purchase counts used in our design. (b) Share of the expected within-minus-between contrast supplied by the 20% most active users when all users are focused ($r = 2$). Bars are exact expectations; black points are means over 100 simulated data sets with 95% intervals. Under low heterogeneity the "active" 20% are a random subset of equally active users.

under COUNT but only $d_u/2$ under WEIGHTED. Figure 2(b) shows what this means in our design. With focused users and the most concentrated activity, the top 20% of users supply 99.6% of the expected COUNT contrast; weighting reduces their share to 84.0%. Shares measured in the simulated data agree to within 0.2 percentage points.

3 Experimental design

Turning the most active users into indiscriminate buyers changes two things. It adds heavy buyers whose pairs carry no category information, and, if the amount of data is held fixed, it takes purchases away from the users who do carry information. No single design can hold both total purchases and informative purchases fixed while doing so, so we use two complementary designs. Design 1, the *matched-total* design, fixes the total number of purchases and asks whether concentrating a fixed amount of activity in a few users helps or hurts each method. Design 2, the *fixed-information* design, fixes the informative users' purchases and adds preference-free heavy buyers on top, which isolates the effect of background noise. Every condition in both designs has 100 independent data sets: 36 conditions in Design 1 and 18 in Design 2, or 5,400 data sets in total. Section A describes the generator step by step.

3.1 Design 1: matched total

We vary four factors (Table 1). *Density* sets the mean number of purchases per user, $m \in \{6, 12\}$. *Activity heterogeneity* determines how those purchases are spread: equally, or with 20% of users buying three or more times the mean. Total purchases are identical across heterogeneity levels, so only their concentration changes. *Preference breadth* is either *focused*, where every user has $r_u = r$, or *broad active*, where the 20% most active users have $r_u = 1$ and

Table 1: Purchase counts per user. Within each density, total purchases are the same in every column.

Mean purchases m	Low heterogeneity	Medium heterogeneity	High heterogeneity
6	all users buy 6	80% buy 3, 20% buy 18	80% buy 1, 20% buy 26
12	all users buy 12	80% buy 6, 20% buy 36	80% buy 2, 20% buy 52

the rest keep $r_u = r$. Under low heterogeneity, where all users are equally active, these are a random 20% of users. Activity is assigned independently of the preferred category. Finally, the *preference strength* $r \in \{2, 3, 4\}$ sets how strongly focused users favor their category. Crossing the four factors gives 36 conditions.

The settings were fixed after a pilot with 10 data sets for each of the 12 conditions at $r = 2$. Following our proposal, we required at least two conditions with a median error between 5% and 40% for some method; four met it at $r = 2$, so we kept the initial design, without regard to which method performed best. We first ran the $r = 2$ grid and then the same grid at $r = 3$ and $r = 4$, the upper value in our proposal, with identical code and fresh seeds.

3.2 Design 2: fixed information

We fix 1,000 informative users in one of two settings ($m = 12$, $r = 2$ or $m = 6$, $r = 3$), then add $A = 50$ or 200 users with no category preference who each buy D products, with $D/m \in \{1, 2, 4, 8\}$, so each added user buys one to eight times as many products as an informative user. Together with a baseline with no added users, this gives nine conditions per setting and 18 in total. Within each replicate the informative purchases are identical across conditions, and the added users and their purchases are nested, so comparisons across conditions isolate the background. Unlike Design 1, total purchases grow as background is added. We added this design after the matched-total results at $r = 2$ showed that adding uninformative heavy buyers and removing informative purchases were confounded.

3.3 Evaluation

Our primary measure is the *misclustering rate*: the fraction of the 300 products assigned to the wrong category after the best matching of cluster labels to categories. To judge how far a method is from chance, we permute the true labels and recompute the matched error, using 20 permutations per data set. We computed this reference for the $r = 3$, $r = 4$, and Design 2 data sets, where it lies between 62.9% and 63.5%; because the band barely moves across these conditions, we also use it as the reference for $r = 2$. All intervals are 95% intervals for a mean over 100 independent data sets. Method comparisons are paired within data sets and are not adjusted for multiple comparisons.

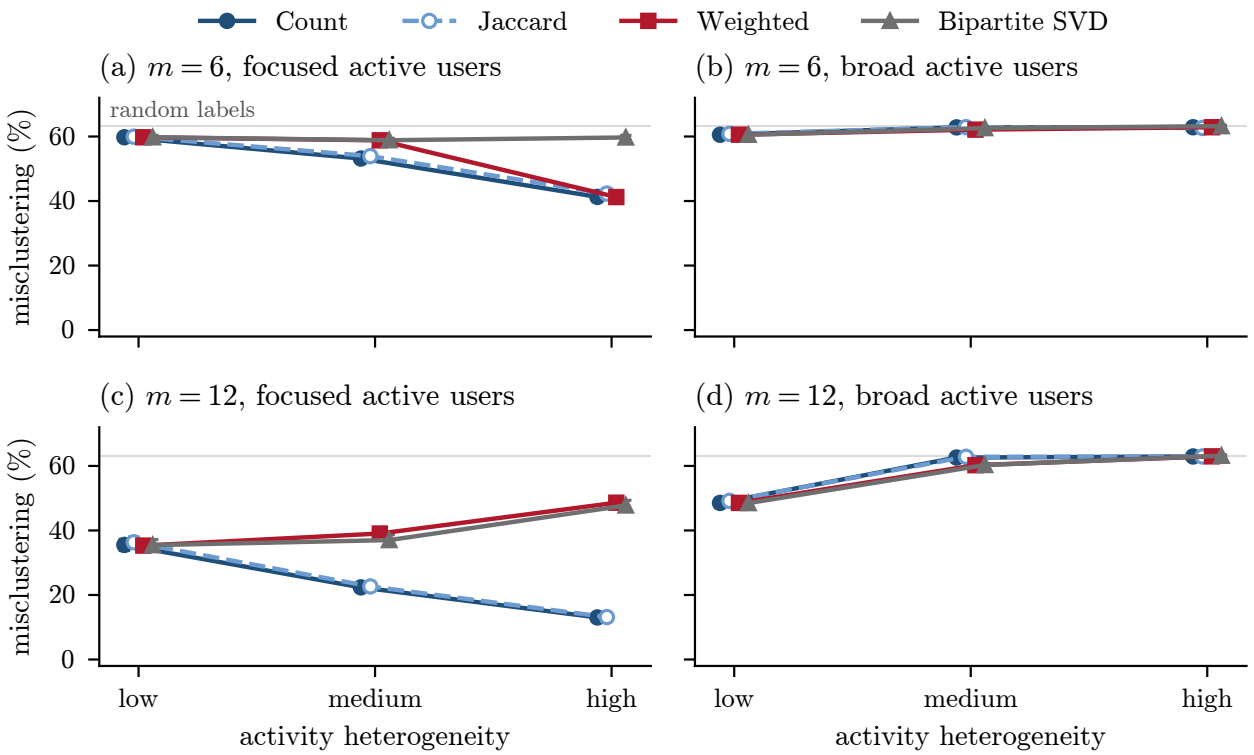


Figure 3: Mean misclustering rate at $r = 2$. Each point averages 100 data sets; error bars are 95% intervals and are often smaller than the markers. The grey band represents the random-label reference.

4 Results

4.1 Design 1 at $r = 2$

Figure 3 summarizes the results at $r = 2$. When active users are focused (left column), concentrating activity helps Count and Jaccard but hurts Weighted and SVD. At $m = 12$, Count error falls from 35.5% under equal activity to 22.4% and then 13.0% as activity becomes more concentrated, while Weighted rises from 35.3% to 39.1% and 48.6%. Heavy buyers who stay within their category are the most informative users in the data, and the count projection lets them speak loudest.

At $m = 6$ the pattern is weaker. At high heterogeneity Count and Weighted coincide (41.3% and 41.2%): the 80% of users with one purchase contribute no pairs, so every contributing user buys 26 products and the two weightings differ by a constant.

When the active users are broad (right column), recovery is poor under medium and high heterogeneity, where every method is within about three points of the random-label reference. With equal activity it is better (48.5% at $m = 12$) because the broad users are no more active than anyone else.

4.2 The paired effect of weighting

Comparing COUNT and WEIGHTED on the same data sets isolates the effect of weighting (all 12 paired differences at $r = 2$ are shown in Figure A2). With focused active users, weighting costs 16.7 percentage points at $m = 12$ and medium heterogeneity (95% interval 15.2 to 18.2) and 35.6 points at high heterogeneity (34.1 to 37.0). Its only clear gains occur with broad active users at medium heterogeneity, 2.4 points at $m = 12$ (1.8 to 3.0) and 0.7 points at $m = 6$, and both leave the error close to chance. Under equal activity the two networks are proportional, so their normalized matrices are identical and the small differences come from k-means initialization.

Weighting does not help in the broad-active conditions, for two different reasons. At $m = 6$ and high heterogeneity, the 800 users with preferences buy one product each and create no pairs, while the 200 heavy buyers have no preference. The expected block contrast is exactly zero under every projection, and no reweighting can create signal that is not there. At $m = 12$, the focused users buy two products each and the broad users 52, so a broad user contributes 1,326 pairs under COUNT and 26 under WEIGHTED, against one for a focused user. Weighting lowers the broad users' share of pair weight from 99.7% to 86.7%, but they still dominate. Because total purchases are matched, making active users broad also takes purchases away from informative users, which is the confound Design 2 removes (Section 4.4).

4.3 Stronger preferences

Raising the preference strength changes the picture (Figure 4; complete results in Section E). With broad active users at $m = 12$ and medium heterogeneity, the methods now separate. COUNT and JACCARD stay at chance, while WEIGHTED falls to 34.8% at $r = 3$ and 9.6% at $r = 4$. The paired advantage of weighting is 27.7 points (26.1 to 29.3) and 51.5 points (50.8 to 52.2). In the focused conditions, stronger preferences make recovery easy for every method, so the cost of weighting shrinks; at $r = 4$ all three projections stay below 5.4% error. At high heterogeneity the broad-active conditions remain at chance for the reason given above.

4.4 Design 2: adding background to fixed informative purchases

Holding the informative purchases fixed separates the two effects (Figure 5). In the first setting, 50 background users buying $4m = 48$ products each push COUNT from 35.5% to 59.1%, whereas WEIGHTED stays at 36.0%; the paired difference is 23.0 points (21.4 to 24.7). In the second setting the difference at $D = 4m$ is 35.2 points (33.9 to 36.5). When $D = m$ the background users are no more active than anyone else, weighting cannot single them out, and the two methods give identical results. Weighting does not make recovery immune to background: with 200 background users, WEIGHTED also loses a few points.

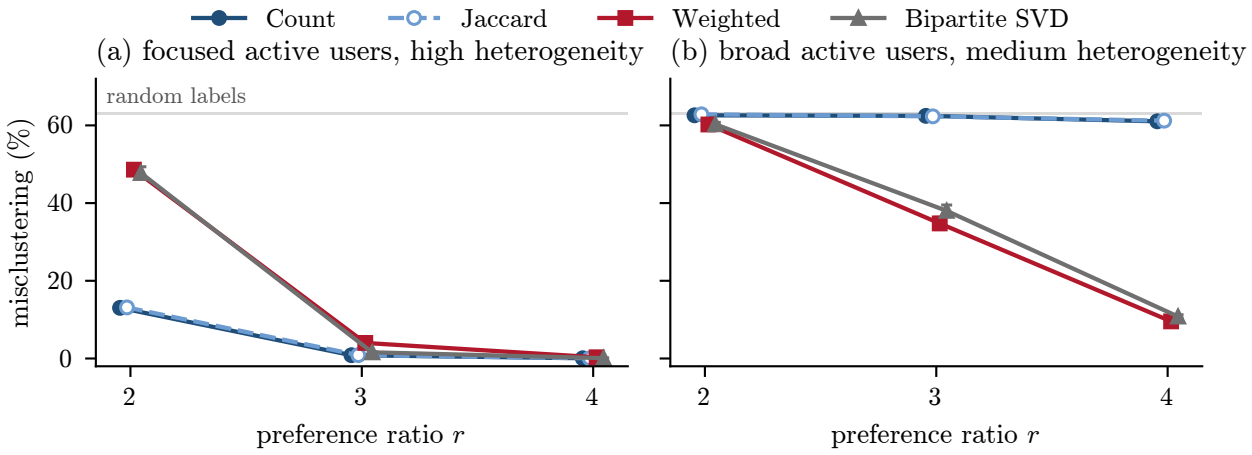


Figure 4: Misclustering rate as preference strength increases, at $m = 12$. Error bars are 95% intervals.

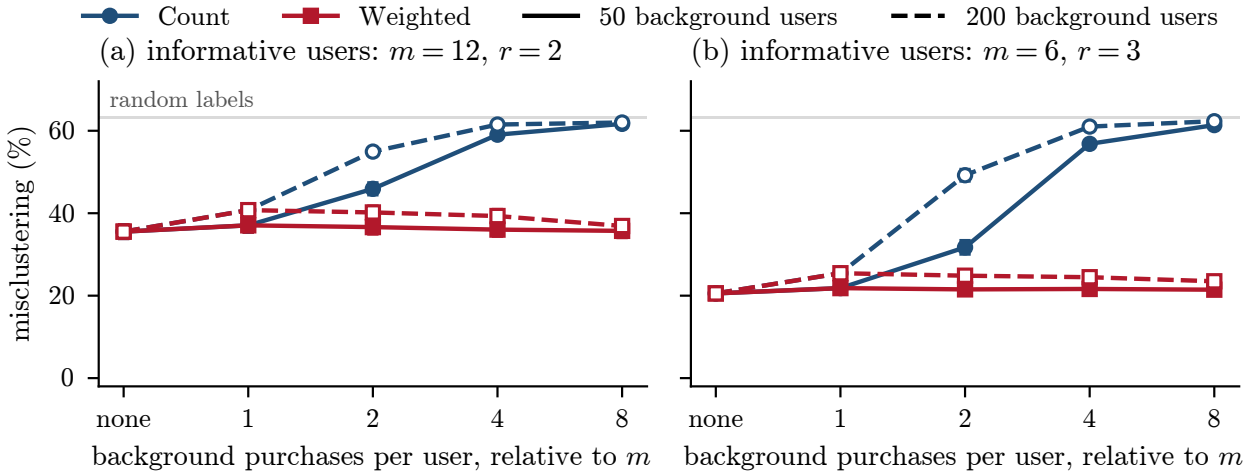


Figure 5: Adding preference-free users to 1,000 fixed informative users: solid lines add 50, dashed lines 200. Jaccard behaves like Count and SVD like Weighted (Section E).

4.5 Jaccard and the bipartite SVD

Count and Jaccard are nearly interchangeable here: their edge weights have a mean Spearman correlation of 0.97 and their mean errors never differ by more than 1.0 point, although their partitions still differ (mean adjusted Rand index 0.70). This is expected when all products in a category are equally popular.

The bipartite SVD tracks Weighted, as (1) predicts, and offers no general advantage: with focused users at $m = 12$ and high heterogeneity its error exceeds that of Count by 34.7 points (33.2 to 36.2), and at $m = 6$ it trails both projections (59.7% against 41%).

Why does the SVD fail at $m = 6$? Users with one purchase contribute nothing to any projection but still enter the SVD through D_u, D_p, and the diagonal of $M^\top M$. In a follow-up check, rerunning the SVD on the recorded data sets without them (after checking that the original errors are reproduced exactly) brings it in line with Count at $m = 6$; at $m = 12$, where no user buys a single product, nothing changes (Table 2).

Table 2: SVD with and without single-purchase users (focused users, high heterogeneity). Mean error (%) over 100 recorded data sets; paired reduction with 95% interval.

Setting	COUNT	WEIGHTED	SVD	SVD, single-purchase users removed	Reduction (points)
$m = 6$, $r = 2$	41.3	41.2	59.7	41.3	18.4 (17.0, 19.9)
$m = 6$, $r = 3$	7.6	7.6	39.3	7.6	31.7 (29.4, 34.0)
$m = 12$, $r = 2$	13.0	48.6	47.8	47.8	0.0

5 Conclusion

Activity is not a proxy for noise. When active users are focused, they carry most of the signal, and inverse-activity weighting increases the error by up to 36 percentage points at $r = 2$. When active users buy at random, weighting helps once the other users carry enough signal: by 28 and 52 points at $r = 3$ and $r = 4$, and by up to 40 points when informative purchases are held fixed. The normalized bipartite SVD behaves much like the weighted projection, as (1) predicts, and offers no general advantage.

In practice, the activity distribution is observable. When activity is nearly even, COUNT and WEIGHTED give essentially the same partition. When a few users dominate, comparing the two partitions indicates which case applies.

6 Limitations and future work

Our model assumes balanced categories, equally popular products, single-category preferences, and known K; degree-corrected or mixed-membership models and real purchase data are natural next steps. Projected edges are dependent, so we do not compare with detection thresholds for independent-edge models. The fixed-information design was added after the matched-total results at $r = 2$ revealed the confound, and we did not study $r < 2$.

References

Inderjit S. Dhillon. Co-clustering documents and words using bipartite spectral graph partitioning. In *Proceedings of the Seventh ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, pages 269–274, 2001. doi: 10.1145/502512.502550.

Z. T. Ke. STAT 236 lecture slides on spectral clustering and network models. Course materials, Harvard University, 2026.

M. E. J. Newman. Scientific collaboration networks. II. Shortest paths, weighted networks, and centrality. *Physical Review E*, 64:016132, 2001. doi: 10.1103/PhysRevE.64.016132.

Karl Rohe, Tai Qin, and Bin Yu. Co-clustering directed graphs to discover asymmetries and directional communities. *Proceedings of the National Academy of Sciences*, 113(45): 12679–12684, 2016. doi: 10.1073/pnas.1525793113.

Ulrike von Luxburg. A tutorial on spectral clustering. *Statistics and Computing*, 17:395–416, 2007. doi: 10.1007/s11222-007-9033-z.

Tao Zhou, Jie Ren, Matúš Medo, and Yi-Cheng Zhang. Bipartite network projection and personal recommendation. *Physical Review E*, 76:046115, 2007. doi: 10.1103/PhysRevE.76.046115.

A Data generation in detail

Every data set in the study is one binary purchase matrix $B \in \{0,1\}^{n \times p}$ together with the true category of each product. This section describes, step by step, how such a matrix is produced in each design, which quantities are fixed and which are random, and how the random seeds are organized.

A.1 Notation and fixed quantities

- $p = 300$ products in $K = 3$ categories of $s = p/K = 100$ products. The categories are contiguous blocks: products 0–99 are category 0, 100–199 category 1, and 200–299 category 2. The true label of product i is z_i. These labels are the same in every data set and are never shown to the methods.
- n users. Design 1 uses $n = 1{,}000$. Design 2 uses 1,000 informative users plus $A \in \{0, 50, 200\}$ background users.
- For user u: the purchase count d_u (the number of *distinct* products bought), the preferred category $g_u \in \{0, 1, 2\}$, and the preference ratio $r_u \geq 1$.
- $B_{ui} = 1$ if user u bought product i. A user buys a product at most once, so the row sums of B are exactly d_u.

A.2 Design 1 (matched total): one data set in six steps

A Design 1 condition is defined by four settings: the mean purchase count $m \in \{6, 12\}$, the activity heterogeneity (low, medium or high), the preference setting (all users focused, or active users broad), and the preference strength $r \in \{2, 3, 4\}$. Given these, one data set is generated as follows.

Step 1: purchase counts. The vector of purchase counts is fixed by the condition; only its assignment to users is random. Under low heterogeneity all users buy m products. Under medium and high heterogeneity, 80% of users buy a small count ℓ and 20% buy a large count $5m - 4\ell$, with $\ell = m/2$ for medium and $\ell = \max(1, m/6)$ for high heterogeneity. This gives the counts in Table A1. Because $(4\ell + (5m - 4\ell))/5 = m$, the mean is exactly m in every column, and the total number of purchases is exactly nm (6,000 or 12,000). The 1,000 counts are then randomly permuted across users.

The same total number of purchases therefore produces very different numbers of product pairs: at $m = 12$, concentrating the purchases in 20% of users quadruples the total COUNT weight, and almost all of the extra weight comes from the heavy buyers.

Step 2: preferred categories. Each user's preferred category g_u is drawn independently and uniformly from $\{0, 1, 2\}$. It is drawn independently of d_u, so heavy and light buyers are equally likely to prefer each category. The three preference groups have about 333 users each,

Table A1: Purchase counts in Design 1. Total purchases are fixed within each value of m. The last column is the total number of product pairs, $\sum_u d_u(d_u-1)/2$, which is the total edge weight of the COUNT projection.

m	Heterogeneity	Purchase counts	Total pairs
6	low	all 1,000 users buy 6	15,000
6	medium	800 buy 3, 200 buy 18	33,000
6	high	800 buy 1, 200 buy 26	65,000
12	low	all 1,000 users buy 12	66,000
12	medium	800 buy 6, 200 buy 36	138,000
12	high	800 buy 2, 200 buy 52	266,000

but their exact sizes vary from data set to data set; this variation is part of the Monte Carlo variability.

Step 3: the active users. The "active" users are the 20% (200 users) with the largest purchase counts. Ties are broken by an independent uniform random number for each user. Under medium and high heterogeneity, exactly 200 users have the large count, so the active set is simply those users. Under low heterogeneity all users are tied, so the active set is a uniformly random 20% of equally active users. This provides a control in which the broad users are no more active than anyone else.

Step 4: preference ratios. Under "all users focused", every user has $r_u = r$. Under "active users broad", the 200 active users have $r_u = 1$, meaning no preference at all, and the remaining 800 users keep $r_u = r$.

Step 5: purchases. Each user's d_u distinct products are drawn one at a time without replacement. Every product i has a fixed weight

$$w_{ui} = \begin{cases} r_u & \text{if } z_i = g_u, \\ 1 & \text{otherwise,} \end{cases}$$

and at each draw a not-yet-purchased product is chosen with probability proportional to its weight among the products still available. The implementation is NumPy's `choice(p, size=d_u, replace=False, p=w_u/sum(w_u))`, which performs exactly this successive sampling. It is neither independent Bernoulli sampling (each product bought independently) nor Fisher's noncentral hypergeometric distribution.

For the first draw, the probability of landing in the preferred category is

$$\frac{r\,s}{r\,s + (p-s)} = \frac{r}{r+K-1},$$

which is 1/3 for $r = 1$, 1/2 for $r = 2$, 3/5 for $r = 3$, and 2/3 for $r = 4$. Later draws deplete the preferred category slightly faster than the others, so the expected share of preferred products decreases a little with d_u (Table A2). The exact distribution of the preferred count is computed by the dynamic program in Section B.

Table A2: Expected share of a user's purchases that fall in the preferred category, computed exactly with the recurrence in Section B.

	$d = 1$	$d = 2$	$d = 6$	$d = 12$	$d = 26$	$d = 52$
$r = 1$	0.333	0.333	0.333	0.333	0.333	0.333
$r = 2$	0.500	0.500	0.498	0.496	0.492	0.483
$r = 3$	0.600	0.600	0.598	0.595	0.587	0.573
$r = 4$	0.667	0.666	0.664	0.660	0.652	0.634

Step 6: output and checks. The generator returns B, the true labels z, the counts d, the preferred categories g, the active indicator, and the ratios r_u. It checks that every row of B sums to d_u. The four methods are then applied to the same B; none of them sees z, g or r_u, and all of them are given the true $K = 3$.

A worked example. Take $m = 12$, high heterogeneity, active users broad, and $r = 4$. After Step 1, 800 users buy 2 products and 200 buy 52. The 200 users buying 52 are the active users, and they get $r_u = 1$, so each of them buys a uniformly random set of 52 products: about $52/3 \approx 17$ per category, carrying no category information. Each of the other 800 users picks two products, each landing in the preferred category with probability about 2/3. A broad user contributes $52 \cdot 51/2 = 1{,}326$ product pairs to the COUNT projection; a focused user contributes one. This is why this condition stays near chance for every method (Section 4.2).

The 36 conditions. Crossing $m \in \{6, 12\}$, three heterogeneity levels, two preference settings and $r \in \{2, 3, 4\}$ gives 36 conditions. Each condition has 100 independent data sets, so Design 1 has 3,600 data sets. The pilot used the same generator with 10 data sets for each of the 12 conditions at $r = 2$.

A.3 Design 2 (fixed information): informative users plus background

Design 2 keeps the informative purchases fixed and adds preference-free heavy buyers on top. A replicate is generated once and then reused for all nine conditions of its setting.

Step 1: informative users. Generate 1,000 users with the Design 1 generator under low heterogeneity and all users focused: every user buys exactly m products with ratio r. There are two settings, $(m, r) = (12, 2)$ and $(m, r) = (6, 3)$. This informative matrix is generated once per replicate and is never modified; a SHA-256 hash of it is stored with every result to confirm that it is identical across conditions.

Step 2: a pool of background purchases. Generate a pool of 200 potential background users. For each, draw a uniformly random ordering of all 300 products and keep the first $8m$ products of the ordering. This is uniform sampling without replacement, the same as $r_u = 1$ in Step 5 above.

Step 3: conditions by nesting. A condition with A background users who each buy D products takes the first A users of the pool and, for each, the first D products of their ordering, and appends these A rows below the informative matrix. The conditions are $A \in \{50, 200\}$ crossed with $D/m \in \{1, 2, 4, 8\}$, plus one baseline with $A = 0$, giving nine conditions per setting and 18 in total, each with 100 replicates (1,800 data sets). Because of the nesting, a condition with more background users or more purchases per background user contains every background purchase of the smaller condition. Differences between conditions within a replicate are therefore caused only by the added background.

How much weight the background carries. Unlike Design 1, total purchases grow with the background: a data set has $1{,}000\,m + AD$ purchases. The informative users contribute $1{,}000\,m(m-1)/2$ pairs to COUNT and $1{,}000\,m/2$ total weight to WEIGHTED; the background contributes $AD(D-1)/2$ and $AD/2$. For example, with $m = 12$, $A = 50$ and $D = 48$, the background supplies 56,400 COUNT pairs against 66,000 informative pairs, but only 1,200 units of WEIGHTED weight against 6,000. When $D = m$, every user buys the same number of products, so the two weightings differ by a constant factor and give identical normalized networks.

A.4 Random seeds

All randomness comes from NumPy generators seeded through `SeedSequence`, so every data set can be regenerated exactly from its recorded seed without storing B.

- **Design 1 at $r = 2$ and the pilot.** A root seed (23603 for $r = 2$, 23601 for the pilot) is spawned into one child seed per data set. Each data set seed is spawned again into five streams: one generates the data, and the other four give the k-means seeds of the four methods.
- **Design 1 at $r = 3$ and $r = 4$.** The data seed is derived from (root, condition index, replicate, 0) and one k-means seed shared by all four methods from (root, condition index, replicate, 1), with roots 23604 and 23607.
- **Design 2.** The informative matrix, the background pool, and the shared k-means seed use separate streams derived from (23605, setting, replicate) with final index 0, 1 and 2. The same k-means seed is used for all methods and all conditions within a replicate.
- **Random-label reference.** The 20 label permutations per data set use the separate root 23606.

Because data generation and clustering use separate streams, changing a method or the number of parallel workers never changes the purchase matrices.

A.5 What the generator deliberately leaves out

The model is simple on purpose, so that expected edge weights can be computed exactly. Categories have equal sizes, and all products within a category are equally popular, which is

why JACCARD has nothing to correct and tracks COUNT. Each user prefers a single category, buys each product at most once, and has a purchase count that is independent of the preferred category. There is no time dimension, no repeat purchase, and no product attribute beyond its category. The number of categories $K = 3$ is known to every method. Results should be read as statements about this controlled model, not about real purchase data.

B Exact category moments

Fix a user with d purchases and preference ratio r, and let $h = p - s$ be the number of non-preferred products. Let $P_t(c)$ be the probability that c of the first t purchases fall in the preferred category, with $P_0(0) = 1$. Given state c after t draws, the next product is preferred with probability

$$a_t(c) = \frac{r(s-c)}{r(s-c) + h - (t-c)},$$

so $P_{t+1}(c+1)$ += $P_t(c)\, a_t(c)$ and $P_{t+1}(c)$ += $P_t(c)\,[1 - a_t(c)]$. After d steps let C be the preferred count and $R = d - C$. Given C, the non-preferred purchases are a uniform R-subset of the h non-preferred products, so the expected number of ordered same-category pairs is

$$A = \mathbb{E}[C(C-1)] + (K-1)\,\mathbb{E}[R(R-1)]\,\frac{s(s-1)}{h(h-1)}.$$

Averaging over a uniformly drawn preferred category gives $q^{\text{same}} = A/[Ks(s-1)]$ and $q^{\text{diff}} = [d(d-1) - A]/[K(K-1)s^2]$.

C Agreement between theory and simulation

Figure A1 compares the expected within-minus-between contrast with its simulated mean in each of the 12 main conditions. For example, at $m = 12$ with focused users and high heterogeneity the expected COUNT contrast is 0.902 and the simulated mean is 0.908; for WEIGHTED the values are 0.0210 and 0.0212.

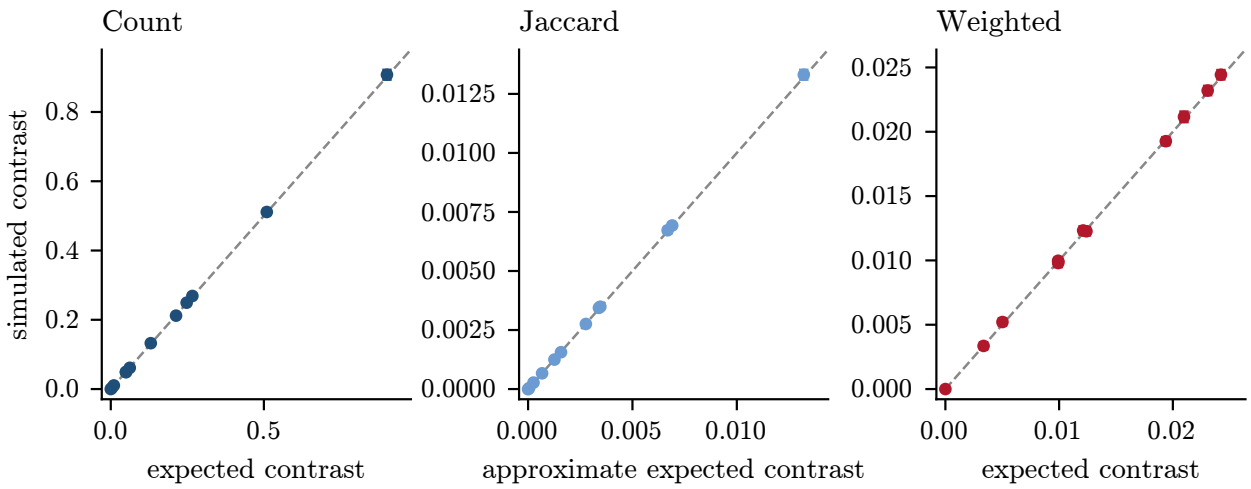


Figure A1: Expected versus simulated within-minus-between contrast, one point per condition, with 95% intervals. The dashed line marks equality. Each panel uses its own units; JACCARD uses the ratio-of-expectations approximation.

D Reproducibility

All simulations use Python 3.11 with the package versions recorded in `requirements-lock.txt`. Root seeds are 23601 (pilot), 23603 ($r = 2$), 23604 ($r = 3$), 23605 (fixed-information design), 23607 ($r = 4$), and 23606 (label permutations). Every data set's seed and k-means seed is stored, so any purchase matrix can be regenerated without storing it. In the $r = 3$, $r = 4$, and fixed-information data sets all methods share one k-means seed, which makes the exact equivalence of COUNT and WEIGHTED under equal activity visible in the results. Unit tests check the design identities, label matching, recovery of a known partition, and the theoretical moments against simulation; validation scripts check every saved summary and paired interval. Mean runtimes per data set were 12.5 ms for COUNT, 12.8 ms for JACCARD, 14.8 ms for WEIGHTED, and 48.2 ms for SVD, excluding data generation. The repository, `https://github.com/diaomj668/236project`, lists the commands that reproduce every result and figure in this report.

E Complete results

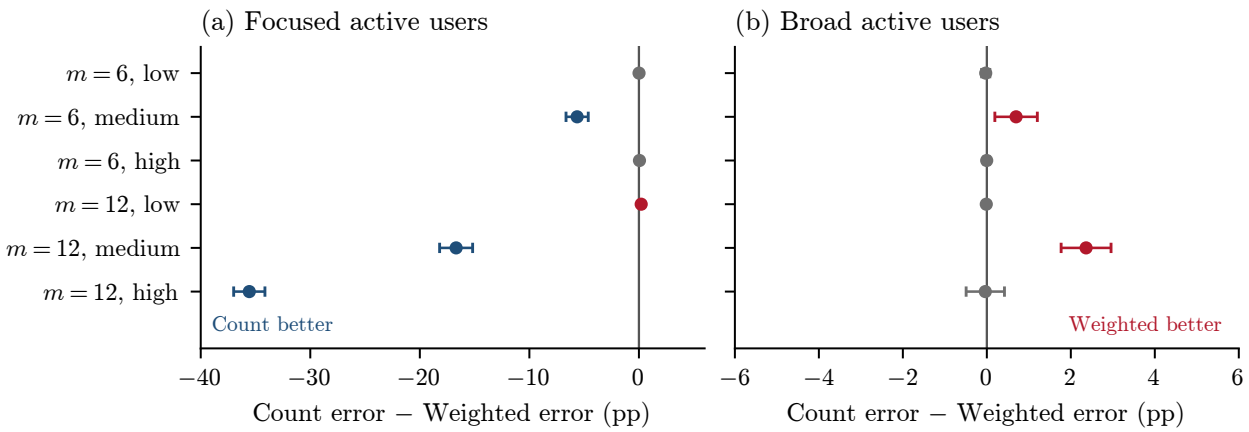


Figure A2: Paired difference in misclustering rate, COUNT minus WEIGHTED, with 95% intervals over 100 data sets. Negative values favour COUNT. Note the different horizontal scales.

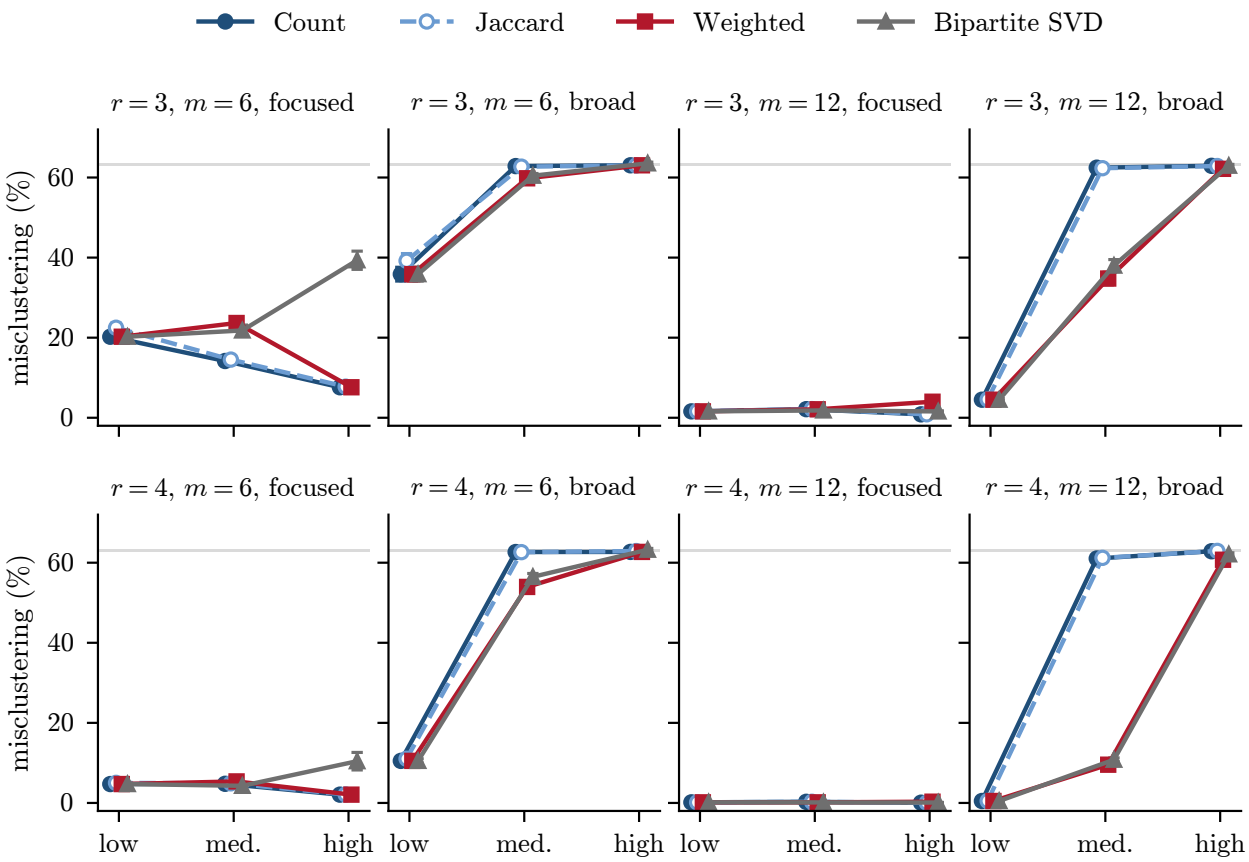


Figure A3: Results at $r = 3$ (top) and $r = 4$ (bottom), with the same layout as Figure 3. The grey band marks the random-label reference.

Table A3: Results at $r = 2$: mean misclustering rate (%) over 100 data sets per condition.

m	Heterogeneity	Active users	COUNT	JACCARD	WEIGHTED	SVD
6	low	focused	59.8	59.9	59.8	59.8
6	low	broad	60.5	60.8	60.6	60.5
6	medium	focused	53.2	53.9	58.8	58.8
6	medium	broad	62.8	62.9	62.1	62.6
6	high	focused	41.3	42.3	41.2	59.7
6	high	broad	62.9	62.6	62.9	63.2
12	low	focused	35.5	36.3	35.3	35.5
12	low	broad	48.5	49.2	48.5	48.5
12	medium	focused	22.4	22.7	39.0	37.0
12	medium	broad	62.6	62.8	60.2	60.3
12	high	focused	13.0	13.1	48.6	47.8
12	high	broad	62.9	62.9	63.0	63.2

Table A4: Results at $r = 3$: mean misclustering rate (%).

m	Heterogeneity	Active users	COUNT	JACCARD	WEIGHTED	SVD
6	low	focused	20.2	22.5	20.2	20.2
6	low	broad	35.9	39.2	35.9	35.9
6	medium	focused	14.2	14.5	23.7	21.8
6	medium	broad	62.9	62.7	59.8	60.5
6	high	focused	7.6	7.8	7.6	39.3
6	high	broad	63.1	63.0	63.1	63.5
12	low	focused	1.6	1.6	1.6	1.6
12	low	broad	4.5	4.6	4.5	4.5
12	medium	focused	2.1	2.2	2.1	1.8
12	medium	broad	62.5	62.3	34.8	38.0
12	high	focused	0.9	0.9	4.0	1.6
12	high	broad	62.9	62.9	62.2	63.0

Table A5: Results at $r = 4$: mean misclustering rate (%).

m	Heterogeneity	Active users	COUNT	JACCARD	WEIGHTED	SVD
6	low	focused	4.7	5.0	4.7	4.7
6	low	broad	10.5	11.0	10.5	10.5
6	medium	focused	4.8	4.9	5.4	4.3
6	medium	broad	62.7	62.6	53.9	56.5
6	high	focused	2.1	2.1	2.1	10.4
6	high	broad	62.7	62.9	62.7	63.2
12	low	focused	0.1	0.1	0.1	0.1
12	low	broad	0.5	0.5	0.5	0.5
12	medium	focused	0.3	0.3	0.1	0.1
12	medium	broad	61.0	61.2	9.6	10.8
12	high	focused	0.1	0.1	0.3	0.1
12	high	broad	62.9	62.9	60.7	62.1

Table A6: Fixed-information design: mean misclustering rate (%) over 100 data sets. A is the number of added preference-free users and D/m their purchases relative to the informative users.

Informative users	A	D/m	COUNT	JACCARD	WEIGHTED	SVD
$m = 12$, $r = 2$	0	–	35.5	36.4	35.5	35.5
	50	1	37.1	37.4	37.1	37.1
	50	2	45.9	46.3	36.6	36.8
	50	4	59.0	59.2	36.0	36.3
	50	8	61.7	61.8	35.7	36.1
	200	1	40.7	41.5	40.7	40.7
	200	2	54.9	55.2	40.2	40.3
	200	4	61.5	61.4	39.3	39.8
	200	8	62.0	62.0	36.9	38.3
$m = 6$, $r = 3$	0	–	20.5	22.3	20.5	20.5
	50	1	21.8	23.5	21.8	21.8
	50	2	31.7	34.4	21.5	21.8
	50	4	56.8	57.3	21.6	22.3
	50	8	61.4	61.6	21.4	22.7
	200	1	25.4	27.3	25.4	25.4
	200	2	49.2	50.2	24.8	26.2
	200	4	61.0	60.9	24.5	27.0
	200	8	62.3	62.4	23.5	26.4